

BookMyTable Code Explanation

File 1: backend/server.js - backend execution start point

This PDF explains the first backend file in an interview-friendly way: what it does, how execution flows, and what answers you can give when asked about the server setup.

One-Line Interview Answer

server.js is the backend bootstrap file. It loads environment variables, creates the Express app, enables middleware, connects to MongoDB, mounts all API routes under /api, adds a global error handler, and starts listening on a port.

Execution Flow

Step	Code Area	Purpose
1	Imports	Loads Express, CORS, dotenv, path helpers, database connection, and routes.
2	__dirname setup	Recreates __dirname because ES modules do not provide it automatically.
3	dotenv.config()	Loads values from backend/.env such as PORT, MongoDB URI, and API keys.
4	express()	Creates the Express application object.
5	Middleware	Enables CORS and parses JSON request bodies.
6	/api/health	Simple route to verify that the backend is running.
7	connectDB()	Initializes the MongoDB connection.
8	app.use('/api', routes)	Mounts all backend API routes under the /api prefix.
9	Error handler	Returns consistent JSON errors instead of exposing raw failures.
10	app.listen()	Starts the server on the configured port.

Important Code Blocks

1. Environment variables

```
dotenv.config({ path: path.join(__dirname, ".env") });
```

This loads backend/.env. In interviews, say that sensitive values such as MongoDB URI and API keys are kept outside source code.

2. Middleware

```
app.use(cors());  
app.use(express.json());
```

cors() allows the React frontend to call the backend from another port. express.json() converts incoming JSON request bodies into req.body.

3. Health check route

```
app.get("/api/health", (req, res) => {  
  res.send("BookMyTable backend is running!");  
});
```

This route is useful for quickly confirming that the backend server is live.

4. Route mounting

```
app.use("/api", routes);
```

Every route exported from routes/index.js receives the /api prefix. For example, if booking routes are mounted as /bookings, the final URL becomes /api/bookings.

5. Global error handler

```
app.use((err, req, res, next) => {  
  console.error("Global Error Log:", err.stack);  
  const statusCode = err.statusCode || 500;  
  res.status(statusCode).json({  
    success: false,  
    message: err.message || "Internal Server Error",  
  });  
});
```

This middleware catches errors passed with next(err) and sends a standard JSON response.

End-To-End Request Example

React frontend

- > POST http://localhost:8082/api/bookings
- > server.js receives the request
- > cors and express.json middleware run
- > routes/index.js forwards the request
- > booking.route.js matches the endpoint
- > booking.controller.js handles business logic
- > booking.model.js saves or reads data in MongoDB
- > JSON response returns to the frontend

Questions Interviewers May Ask

Question	Best Short Answer
Why use cors?	Because frontend and backend may run on different origins, such as different localhost ports.
Why use express.json()?	To parse JSON sent by the frontend and make it available inside req.body.
Why use dotenv?	To keep configuration and secrets outside the codebase.
What does app.use('/api', routes) mean?	It prefixes all imported routes with /api.
What is the global error handler for?	It centralizes error responses and avoids repeating error response code in every controller.
What happens when app.listen runs?	The Express app starts accepting HTTP requests on the selected port.

Final Interview Script

In BookMyTable, server.js is the starting point of the backend. It loads environment variables with dotenv, creates an Express app, applies CORS and JSON middleware, exposes a health-check route, connects to MongoDB, and mounts all routes under /api. When a frontend request comes in, middleware runs first, then the request is passed to the route layer, then to the controller, and finally the controller talks to the MongoDB model. Any errors are handled by a global error middleware, and app.listen starts the backend on the configured port.